

CSE425 / EEE474 - Neural Networks

Autoencoders / Variational Autoencoders / LSTM Autoencoders Math Problems

Part I: Standard Autoencoders

Problem 1 - Architecture and reconstruction loss

A fully connected autoencoder for MNIST images (28×28 pixels, flattened to $D = 784$) uses encoder layers $784 \rightarrow 300 \rightarrow 100 \rightarrow 2$ and symmetric decoder layers $2 \rightarrow 100 \rightarrow 300 \rightarrow 784$.

- (a) State the dimension of the latent vector z .
- (b) Write the mean squared reconstruction loss $L(x, x')$.
- (c) Identify which part of the network compresses the input and which reconstructs it.

Solution

- (a) $z \in \mathbb{R}^2$. The autoencoder squeezes a 784-dimensional image all the way down to just 2 numbers. Those two numbers are the latent code.

(b)

$$L(x, x') = \frac{1}{N} \sum_{n=1}^N (x^{(n)} - x'^{(n)})^2$$

This measures how different the reconstructed image x' is from the original input x , pixel by pixel. Training drives this difference towards zero.

- (c) The **encoder** ($784 \rightarrow 300 \rightarrow 100 \rightarrow 2$) compresses the input: it reads 784 pixel values and produces a 2-number summary z . The **decoder** ($2 \rightarrow 100 \rightarrow 300 \rightarrow 784$) does the reverse: it reads the 2-number summary and tries to recreate all 784 pixel values.

Problem 2 - Trainable parameter count

For the autoencoder in Problem 1, assume every layer is fully connected with bias terms. A layer with m inputs and n outputs has $m \times n$ weight parameters plus n bias parameters, giving $mn + n$ in total.

Compute (a) the encoder parameter count, (b) the decoder parameter count, and (c) the total for the full autoencoder.

Solution

- (a) Encoder layers, counted one by one:

$$784 \rightarrow 300 : 784 \times 300 + 300 = 235,500$$

$$300 \rightarrow 100 : 300 \times 100 + 100 = 30,100$$

$$100 \rightarrow 2 : 100 \times 2 + 2 = 202$$

Encoder total: $235,500 + 30,100 + 202 = \mathbf{265,802}$.

- (b) Decoder layers (mirror image of the encoder):

$$2 \rightarrow 100 : 2 \times 100 + 100 = 300$$

$$100 \rightarrow 300 : 100 \times 300 + 300 = 30,300$$

$$300 \rightarrow 784 : 300 \times 784 + 784 = 235,984$$

Decoder total: $300 + 30,300 + 235,984 = \mathbf{266,584}$.

- (c) Full autoencoder: $265,802 + 266,584 = \mathbf{532,386}$ parameters.

Problem 3 - Numerical forward pass

A small autoencoder has a 2-dimensional input and a 1-dimensional latent space. The encoder is a single linear layer with weights $w = (0.6, 0.4)$ (no bias) followed by ReLU. The decoder is a single linear layer with weights $v = (0.5, 0.9)$ (no bias).

Given $x = (1.0, 2.0)$:

- (a) Compute the latent code z after ReLU.
- (b) Compute the reconstruction $x' = (x'_1, x'_2)$.
- (c) Compute the reconstruction loss $L = (x_1 - x'_1)^2 + (x_2 - x'_2)^2$.

Solution

- (a) The encoder takes the dot product of w with x , then applies ReLU:

$$h = 0.6 \times 1.0 + 0.4 \times 2.0 = 0.6 + 0.8 = 1.4.$$

$\text{ReLU}(1.4) = \max(0, 1.4) = \mathbf{1.4}$. Since $h > 0$, the ReLU passes it through unchanged.

- (b) The decoder scales the single latent number $z = 1.4$ by each decoder weight:

$$x'_1 = 0.5 \times 1.4 = \mathbf{0.70}, \quad x'_2 = 0.9 \times 1.4 = \mathbf{1.26}.$$

- (c) Comparing original vs. reconstructed, dimension by dimension:

$$\begin{aligned} L &= (1.0 - 0.70)^2 + (2.0 - 1.26)^2 \\ &= (0.30)^2 + (0.74)^2 \\ &= 0.09 + 0.5476 = \mathbf{0.6376}. \end{aligned}$$

The reconstruction is imperfect because a single number z cannot capture all information

in a 2-dimensional input. Training would adjust w and v to reduce this loss.

Problem 4 - Why the decoder is discarded after training

After training a standard autoencoder on MNIST, a data scientist wants to use the model for handwritten digit classification. She keeps only the encoder and discards the decoder.

- (a) What does the encoder provide, and why is it useful for classification? Give a concrete example using the 2D latent space from Problem 1.
- (b) Why is the decoder not useful for the classification task?

Solution

- (a) The encoder compresses each 784-dimensional image into a 2-number vector $z = (z_1, z_2)$. Because the autoencoder was trained to reconstruct digits, these 2 numbers encode the most important visual structure. Images of the same digit tend to produce similar z vectors and cluster together in 2D space. For example, all images of the digit “1” might land near $z \approx (-0.6, 0.3)$, while “5” lands near $z \approx (0.4, -0.5)$. A simple classifier (such as a k -nearest-neighbour or a small linear layer) can then separate the ten digit classes using just these 2 numbers instead of 784. This is much faster and often generalises better because noise and irrelevant variation have been discarded.
- (b) The decoder turns a latent code z back into a pixel-level image. For classification we do not need a reconstructed image at all; we need to know *which digit* the image shows. The decoder output is a 784-dimensional vector of pixel intensities, not a class label. It adds computation without helping answer the classification question, so it is discarded.

Part II: LSTM Autoencoder for Sequence Denoising

Problem 5 - Architecture review

An LSTM autoencoder processes noisy time series data. The input shape is (batch, 50, 2): each sample is a sequence of 50 timesteps, each with 2 measured features. The encoder LSTM has 100 hidden units; the decoder LSTM also has 100 hidden units.

- (a) What does the encoder output after processing all 50 timesteps? What does this vector represent?
- (b) Why is a `RepeatVector(50)` layer needed between encoder and decoder? Give its output shape.
- (c) What does `TimeDistributed(Dense(2))` do, and why is it needed?
- (d) Write the training loss L for the denoising task.

Solution

- (a) After reading all 50 timesteps one at a time, the encoder LSTM produces its final hidden state $z = h_{50} \in \mathbb{R}^{100}$. This single 100-dimensional vector is a compressed summary (the bottleneck) of the entire noisy sequence. Think of it as the encoder’s

answer to the question: “what is the essential pattern in this sequence?”

- (b) The decoder LSTM needs *one input vector per timestep* to produce 50 output timesteps. But the encoder gave us only one vector z . The `RepeatVector(50)` layer solves this by copying z exactly 50 times, creating a tensor of shape (batch, 50, 100). Every decoder timestep therefore starts from the same summary of the sequence.
- (c) After the decoder LSTM runs, it produces a 100-dimensional hidden state h'_t at each of the 50 timesteps. But the original data has only 2 features per timestep, not 100. The `TimeDistributed(Dense(2))` layer applies the same small linear projection ($100 \rightarrow 2$) independently at every timestep, converting each $h'_t \in \mathbb{R}^{100}$ into the reconstructed output $\hat{x}_t \in \mathbb{R}^2$.
- (d) Mean squared error between the model output and the *clean* signal:

$$L = \frac{1}{T} \sum_{t=1}^T \|x_t - \hat{x}_t\|^2.$$

Crucially, the encoder receives the *noisy* input $\tilde{x}_t = x_t + \varepsilon_t$, but the loss is measured against the clean x_t . The model therefore learns to remove the noise.

Problem 6 - Parameter count for the LSTM autoencoder

For the LSTM autoencoder in Problem 5, compute the number of trainable parameters in (a) the encoder LSTM (input size 2, hidden size 100), (b) the decoder LSTM (input size 100, hidden size 100), (c) the `TimeDistributed(Dense(2))` output layer, and (d) verify the total of 121,802 from the course slides.

Formula: an LSTM with input size n and hidden size d has four gates (forget, input, candidate, output), each requiring a weight matrix for the current input ($n \times d$), a weight matrix for the previous hidden state ($d \times d$), and a bias vector (d). Total: $4(nd + d^2 + d)$.

Solution

- (a) Encoder LSTM ($n = 2, d = 100$). Each gate needs a 2×100 input matrix, a 100×100 recurrent matrix, and 100 biases:

$$4(2 \times 100 + 100 \times 100 + 100) = 4(200 + 10,000 + 100) = 4 \times 10,300 = \mathbf{41,200}.$$

- (b) Decoder LSTM ($n = 100, d = 100$). Now the input is also 100-dimensional:

$$4(100 \times 100 + 100 \times 100 + 100) = 4(10,000 + 10,000 + 100) = 4 \times 20,100 = \mathbf{80,400}.$$

- (c) `TimeDistributed(Dense(2))`: a single layer mapping 100 inputs to 2 outputs, with bias:

$$100 \times 2 + 2 = \mathbf{202}.$$

- (d) Total: $41,200 + 80,400 + 202 = \mathbf{121,802}$.

Problem 7 - Why the bottleneck removes noise

The encoder bottleneck is $z = h_{50} \in \mathbb{R}^{100}$. The input sequence has 50 timesteps with 2 features each. Explain in plain terms why this bottleneck forces the network to keep signal and throw away noise. Your answer should mention (a) why noise cannot be compressed efficiently, (b) the role of the fixed bottleneck size, and (c) how the LSTM cell state reinforces the effect.

Include a brief numerical illustration of how a noise spike decays through successive forget gates (use $f_t = 0.1$ as an example).

Solution

(a) Why noise cannot be compressed. The latent vector $z \in \mathbb{R}^{100}$ must summarise an entire sequence of 50 timesteps each with 2 features. The actual signal (e.g., a sine wave) follows a smooth, predictable pattern that can be described compactly in 100 numbers. Noise, on the other hand, is random and unpredictable at every timestep; there is no short description for 50 independent random values. The encoder therefore *has* to focus on the signal, because it is the only thing that fits in the bottleneck.

(b) The bottleneck enforces compression. If the bottleneck were very large (say, 10,000 dimensions), the encoder could afford to memorise the noise as well. With only 100 dimensions the encoder is forced to be selective; random fluctuations are simply not worth the bits.

(c) The LSTM cell state smooths out spikes. At each timestep the forget gate f_t decides what fraction of the past cell state to carry forward. A single noisy spike at timestep t contributes to the cell state c_t , but subsequent forget gates attenuate it. With $f_t = 0.1$ (the gate mostly forgets), the spike's contribution after each additional step is:

Steps after spike	Remaining influence
1	$0.1^1 = 0.1000$
2	$0.1^2 = 0.0100$
3	$0.1^3 = 0.0010$
4	$0.1^4 = 0.0001$
5	$0.1^5 \approx 0.000010$

After just a few steps the spike has almost no effect on the final hidden state h_{50} . A smooth signal, by contrast, reinforces itself at every step and survives to the bottleneck.

Part III: Variational Autoencoder

Problem 8 - Standard autoencoder vs. VAE encoder output

For a given input x , compare what the encoder outputs in (a) a standard autoencoder and (b) a VAE. Then (c) explain in simple terms why the VAE encoder must output two vectors rather than one, and what problem this solves.

Solution

- (a) A standard autoencoder encoder outputs a single fixed point $z \in \mathbb{R}^K$, like a GPS coordinate. The same input always produces exactly the same code.
- (b) A VAE encoder outputs two vectors: a mean $\mu \in \mathbb{R}^K$ and a log-variance $\log \sigma^2 \in \mathbb{R}^K$. Together they define a Gaussian *cloud* of possible codes around μ , with spread σ^2 . Each forward pass samples a different point from this cloud.
- (c) In a standard autoencoder, the latent space looks like a scattered map of isolated islands (one point per training image). If you sample a random point between two islands, the decoder has never seen that region before and produces garbage.

The VAE forces every encoded image to be represented by a *cloud* rather than a point, and the KL loss pushes all these clouds towards the centre of the space. This fills in the gaps: every point in the latent space is now near some training-time distribution, so the decoder can handle any sample from the prior. This is what makes the VAE a proper generative model.

Problem 9 - The reparameterization trick: concept and numerical example

A VAE encoder produces $\mu_\phi(x) = 0.3$ and $\log \sigma_\phi^2(x) = -0.5$. A noise draw from $\mathcal{N}(0, 1)$ gives $\varepsilon = -0.2$.

- (a) Compute σ from $\log \sigma^2 = -0.5$. Show the step.
- (b) Compute z using the reparameterization formula $z = \mu + \sigma \times \varepsilon$.
- (c) Suppose instead we tried to sample z directly from $\mathcal{N}(\mu, \sigma^2)$ without any trick. Explain in simple terms why this breaks backpropagation.
- (d) Explain how the reparameterization trick fixes the problem. What is the key insight about where the randomness now lives?
- (e) Write down $\partial z / \partial \mu$ and $\partial z / \partial \sigma$ for the reparameterised formula $z = \mu + \sigma \varepsilon$. Why do these matter?

Solution

- (a) The network outputs $s = \log \sigma^2 = -0.5$. Recovering σ :

$$\sigma = \exp\left(\frac{s}{2}\right) = \exp\left(\frac{-0.5}{2}\right) = \exp(-0.25) \approx \mathbf{0.7788}.$$

- (b)

$$z = 0.3 + 0.7788 \times (-0.2) = 0.3 - 0.1558 \approx \mathbf{0.1442}.$$

- (c) Imagine backpropagation as a chain of instructions flowing backwards from the loss to each parameter. The instruction at every step is “how much does the loss change if I nudge this number by a tiny amount?” (that is, a gradient). When z is *drawn randomly* from $\mathcal{N}(\mu, \sigma^2)$, changing μ by a tiny amount does not change the sample in any deterministic way; it just shifts the probability of drawing each value. There is no clean derivative. The gradient cannot flow backwards through a random draw, so the encoder parameters μ and σ receive no update signal at all.

- (d) The reparameterization trick separates the deterministic part from the random part:

$$z = \mu + \sigma \times \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, 1).$$

The key insight: ε is *fixed* during the backward pass. It is sampled once before the pass begins and treated as a constant input, like a data point. The randomness now lives in ε , which has no parameters to update. The trainable quantities μ and σ appear in a plain arithmetic formula, so gradients can flow through them normally.

- (e)

$$\frac{\partial z}{\partial \mu} = 1, \quad \frac{\partial z}{\partial \sigma} = \varepsilon = -0.2.$$

These are the gradients that backpropagation needs to update the encoder. $\partial z / \partial \mu = 1$ says: if the loss increases when z increases, then μ should decrease by the same amount. Without the trick these gradients would not exist, and the encoder could not learn anything from the reconstruction loss.

Problem 10 - The log-variance parameterization

VAE encoders output $s = \log \sigma^2$ rather than σ directly.

- In simple terms, why is this better than asking the network to output σ directly?
- Show how to recover σ from s .
- Given $s = -1.2$, $\mu = 0.5$, and $\varepsilon = 1.0$, compute σ and z .
- Complete the table below. Confirm that $\sigma > 0$ for all values of s , including negative ones.

$s = \log \sigma^2$	$\sigma = \exp(s/2)$
-2.0	
-0.5	
0.0	
1.0	

Solution

- Standard deviation σ must always be positive ($\sigma > 0$). If the network tried to output σ directly, we would need a special activation function (like softplus) to keep the output positive, and very small values near zero would cause numerical instability. Instead, the network outputs $s = \log \sigma^2$, which can be any real number, positive or negative. Any real output from the network corresponds to a valid positive σ via $\sigma = \exp(s/2)$. The network can therefore use an ordinary linear output layer with no activation, which is more stable to train.
- From $s = \log \sigma^2$ we get $\sigma^2 = e^s$, so:

$$\sigma = \sqrt{e^s} = e^{s/2} = \exp(s/2).$$

(c) $\sigma = \exp(-1.2/2) = \exp(-0.6) \approx \mathbf{0.5488}$.

$$z = 0.5 + 0.5488 \times 1.0 = \mathbf{1.0488}.$$

	$s = \log \sigma^2$	$\sigma = \exp(s/2)$
	-2.0	$\exp(-1.0) \approx 0.3679$
(d)	-0.5	$\exp(-0.25) \approx 0.7788$
	0.0	$\exp(0.0) = 1.0000$
	1.0	$\exp(0.5) \approx 1.6487$

Even when s is very negative (e.g., $s = -2.0$), $\sigma = 0.3679$ is still positive. The exponential function guarantees $\sigma > 0$ for all real s .

Problem 11 - Full VAE forward pass: toy setup $D = 2, d = 1$

A VAE has a 2-dimensional input $\mathbf{x} = (x_1, x_2)$ and a single latent dimension $z \in \mathbb{R}$. The prior is $p(z) = \mathcal{N}(0, 1)$. The decoder applies sigmoid to produce $\hat{\mathbf{x}} \in (0, 1)^2$, so the data is treated as binary (each pixel is either on or off).

The encoder produces: $\mu = 0.3, \log \sigma^2 = -0.5$.

Noise draw: $\varepsilon = -0.2$.

Decoder output: $\hat{x}_1 = 0.7, \hat{x}_2 = 0.4$.

True data: $x_1 = 1$ (pixel on), $x_2 = 0$ (pixel off).

- Compute σ and the latent sample z .
- Compute the Bernoulli reconstruction term:

$$\log p(\mathbf{x}|z) = \sum_{i=1}^2 [x_i \log \hat{x}_i + (1 - x_i) \log(1 - \hat{x}_i)].$$

Explain what each term in the sum is doing.

- Compute the KL term:

$$D_{\text{KL}} = \frac{1}{2}(\sigma^2 + \mu^2 - \log \sigma^2 - 1).$$

- Compute the ELBO and the final loss that is minimised during training. Explain the sign.

Solution

(a) $\sigma = \exp(-0.5/2) = \exp(-0.25) \approx 0.7788$.

$$z = 0.3 + 0.7788 \times (-0.2) = 0.3 - 0.1558 \approx \mathbf{0.1442}.$$

(b) Since $x_1 = 1$ and $x_2 = 0$:

$$\begin{aligned}\log p(\mathbf{x}|z) &= [1 \cdot \log 0.7 + 0 \cdot \log 0.3] + [0 \cdot \log 0.4 + 1 \cdot \log(1 - 0.4)] \\ &= \log 0.7 + \log 0.6 \\ &\approx -0.3567 + (-0.5108) = -\mathbf{0.8675}.\end{aligned}$$

What each term does. For $x_1 = 1$ (the pixel should be on), the term $\log \hat{x}_1 = \log 0.7$ rewards the decoder for predicting $\hat{x}_1 = 0.7$ (close to 1). For $x_2 = 0$ (the pixel should be off), the term $\log(1 - \hat{x}_2) = \log 0.6$ rewards the decoder for predicting $\hat{x}_2 = 0.4$ (reasonably close to 0). A perfect reconstruction would give $\log 1 + \log 1 = 0$; wrong predictions produce large negative values.

(c) Using $\sigma^2 = \exp(-0.5) \approx 0.6065$ and $\log \sigma^2 = -0.5$:

$$D_{\text{KL}} = \frac{1}{2}(0.6065 + 0.09 - (-0.5) - 1) = \frac{1}{2}(0.6065 + 0.09 + 0.5 - 1) = \frac{1}{2}(0.1965) \approx \mathbf{0.0983}.$$

(d)

$$\text{ELBO} = \log p(\mathbf{x}|z) - D_{\text{KL}} = -0.8675 - 0.0983 \approx -\mathbf{0.9658}.$$

$$\text{Loss to minimise} = -\text{ELBO} \approx \mathbf{0.9658}.$$

The ELBO is negative because $\log p(\mathbf{x}|z) \leq 0$ for any probability below 1. Training maximises the ELBO, which is the same as minimising its negation. A loss of 0.9658 means the model is not yet reconstructing confidently; a well-trained model would push this towards zero.

Problem 12 - VAE loss function and numerical KL evaluation

Write the complete VAE training objective (to minimise) for real-valued data using MSE as the reconstruction term. Then compute the KL regularisation loss for a VAE with $K = 3$ latent dimensions whose encoder outputs:

$$\boldsymbol{\mu} = (0.5, -1.0, 0.2), \quad \boldsymbol{\sigma} = (1.2, 0.5, 0.8).$$

Show your working dimension by dimension and identify which dimension contributes most. Give your final answer to four decimal places.

Solution

The VAE training objective is:

$$\min_{\theta, \phi} \underbrace{\sum_{i=1}^D (x_i - \hat{x}_i)^2}_{\text{reconstruction loss (MSE)}} + \underbrace{\frac{1}{2} \sum_{j=1}^K (\sigma_j^2 + \mu_j^2 - \log \sigma_j^2 - 1)}_{\text{KL regularisation (keeps latent space organised)}}.$$

The reconstruction term pushes the decoder to reproduce x faithfully. The KL term pushes the encoder distribution towards the standard normal prior, so that random samples at

generation time and in regions the decoder can decode.

Computing each per-dimension contribution $c_j = \sigma_j^2 + \mu_j^2 - \log \sigma_j^2 - 1$:

j	μ_j	σ_j	σ_j^2	$\log \sigma_j^2$	c_j
1	0.5	1.2	1.44	$\ln 1.44 \approx 0.3646$	$1.44 + 0.25 - 0.3646 - 1 = 0.3254$
2	-1.0	0.5	0.25	$\ln 0.25 \approx -1.3863$	$0.25 + 1.00 + 1.3863 - 1 = 1.6363$
3	0.2	0.8	0.64	$\ln 0.64 \approx -0.4463$	$0.64 + 0.04 + 0.4463 - 1 = 0.1263$
$D_{\text{KL}} = \frac{1}{2}(0.3254 + 1.6363 + 0.1263) = \frac{1}{2}(2.0880) = \mathbf{1.0440}.$					

The largest contributor is dimension $j = 2$: the mean $\mu_2 = -1.0$ is far from zero, which the KL term penalises. Training would push μ_2 closer to zero and σ_2 closer to 1, reducing this KL penalty.

Problem 13 - Cross-entropy is not symmetric

The binary cross-entropy between a true distribution p and a predicted distribution q is:

$$H(p, q) = - \sum_i p_i \log q_i.$$

Let $p = (0.8, 0.2)$ and $q = (0.7, 0.3)$.

- Compute $H(p, q)$ step by step.
- Compute $H(q, p)$ step by step.
- Are the two values equal? Explain in simple terms what the asymmetry means and why it matters for training a VAE.
- In VAE training with binary pixel data, which distribution is p and which is q ? Explain why getting this the right way round is important.
- Give a concrete pixel-level numerical example showing that a correct prediction is penalised less than a wrong one.

Solution

(a)

$$\begin{aligned}
 H(p, q) &= -(p_1 \log q_1 + p_2 \log q_2) \\
 &= -(0.8 \times \log 0.7 + 0.2 \times \log 0.3) \\
 &= -(0.8 \times (-0.3567) + 0.2 \times (-1.2040)) \\
 &= -(-0.2854 - 0.2408) \\
 &= \mathbf{0.5261}.
 \end{aligned}$$

(b)

$$\begin{aligned} H(q, p) &= -(q_1 \log p_1 + q_2 \log p_2) \\ &= -(0.7 \times \log 0.8 + 0.3 \times \log 0.2) \\ &= -(0.7 \times (-0.2231) + 0.3 \times (-1.6094)) \\ &= -(-0.1562 - 0.4828) \\ &= \mathbf{0.6390}. \end{aligned}$$

(c) $H(p, q) = 0.5261 \neq 0.6390 = H(q, p)$, so cross-entropy is **not symmetric**. The intuition: $H(p, q)$ uses p as the weighting. The term for outcome 1 gets weight $p_1 = 0.8$ (high) and the term for outcome 2 gets weight $p_2 = 0.2$ (low). So the penalty is dominated by how well the model predicts outcome 1, which is the truly likely outcome. Swapping to $H(q, p)$ changes the weights to $q_1 = 0.7$ and $q_2 = 0.3$, giving a different penalty that no longer accurately reflects which outcome actually matters. In VAE training we must put the true data in the p position to ensure the loss correctly weights errors on likely pixels.

(d) $p = \mathbf{x}$ (the true pixel values) and $q = \hat{\mathbf{x}}$ (the decoder output after sigmoid). We minimise $H(\mathbf{x}, \hat{\mathbf{x}})$. Putting true data in the p position means the gradient of the loss pushes $\hat{\mathbf{x}}$ towards $\mathbf{x}$, which is the correct learning direction. Reversing to $H(\hat{\mathbf{x}}, \mathbf{x})$ would evaluate $\log p_i = \log x_i$ where $x_i \in \{0, 1\}$; when $x_i = 0$, $\log 0$ is undefined, so the formula breaks entirely.

(e) Consider a single pixel with true value $x = 1$ (should be white):

$$\hat{x} = 0.9 \text{ (almost correct): } -[1 \cdot \log 0.9 + 0 \cdot \log 0.1] = -\log 0.9 \approx \mathbf{0.1054}.$$

$$\hat{x} = 0.1 \text{ (very wrong): } -[1 \cdot \log 0.1 + 0 \cdot \log 0.9] = -\log 0.1 \approx \mathbf{2.3026}.$$

A near-correct prediction is penalised only 0.11; a near-wrong prediction is penalised 2.30, more than 20 times as much. The logarithm makes errors on confident wrong predictions extremely costly.

Problem 14 - Which component to keep after training

A VAE is trained on MNIST. A researcher has two separate tasks:

- (i) embed new test images into a structured latent space for visualisation, and
- (ii) generate brand new handwritten digit images that were never in the training set.
 - (a) For task (i), which component is used and which is discarded? Why?
 - (b) For task (ii), which component is used, what is fed as input, and why is the *encoder not needed*?
 - (c) Standard autoencoders are used differently: what is typically kept there, and why does the role of each component differ between the two models?

Solution

- (a) For embedding (task i), only the **encoder** is used; the decoder is discarded. The encoder maps each image $\mathbf{x}$ to its mean code $\boldsymbol{\mu}_\phi(\mathbf{x})$, placing it in the 2D (or K -D) latent space. Images of the same digit cluster together there, making the space useful for visualisation and downstream analysis. The decoder is irrelevant because producing a pixel-level image is not the goal.
- (b) For generation (task ii), only the **decoder** is used; the encoder is discarded. The input is a random sample $z \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ drawn from the prior. The decoder maps this z to a new image $\hat{\mathbf{x}}$.

The encoder is not needed because there is no real image to encode. The whole point is to *create* an image that does not already exist. The KL training objective has already shaped the latent space so that any sample from $\mathcal{N}(\mathbf{0}, \mathbf{I})$ lands in a region the decoder was trained to decode coherently.

- (c) In a standard autoencoder, the **encoder** is kept and the decoder is discarded, for downstream tasks like classification. The encoder gives a compact, meaningful feature vector z for each image. The decoder can only reconstruct pixels, which is useless for classification.

The roles differ because the two models have different latent spaces: the standard autoencoder latent space is unstructured (scattered islands), so random sampling from it produces garbage. The VAE latent space is smooth and continuous, so random sampling produces valid images. This is what makes the VAE decoder worth keeping for generation, and what makes the standard autoencoder decoder worthless for the same task.

Problem 15 - Mini-batch VAE loss with numerical verification

A mini-batch of $M = 2$ samples passes through a VAE with $K = 1$ latent dimension and binary outputs. The encoder and decoder produce the following values.

m	$\mu^{(m)}$	$\log \sigma^{2(m)}$	$\sigma^{(m)}$	$\varepsilon^{(m)}$	$z^{(m)}$	$x^{(m)}$	$\hat{x}^{(m)}$
1	0.3	-0.5	0.7788	-0.2	0.1442	1.0	0.7
2	-0.4	0.2	1.1052	0.5	0.1526	0.0	0.3

Per-sample binary cross-entropy reconstruction loss: $R^{(m)} = -[x^{(m)} \log \hat{x}^{(m)} + (1 - x^{(m)}) \log(1 - \hat{x}^{(m)})]$.

- (a) Verify $\sigma^{(m)}$ and $z^{(m)}$ for both samples using the log-variance formula and the reparameterization trick.
- (b) Compute $R^{(m)}$ for each sample and explain why both values are the same even though $x^{(1)} = 1$ and $x^{(2)} = 0$.
- (c) Compute $D_{\text{KL}}^{(m)} = \frac{1}{2}(\sigma^{2(m)} + \mu^{2(m)} - \log \sigma^{2(m)} - 1)$ for each sample.
- (d) Compute the mini-batch loss $\mathcal{L} = \frac{1}{M} \sum_{m=1}^M (R^{(m)} + D_{\text{KL}}^{(m)})$ and explain why we average over M rather than sum.

Solution

(a) Verification using $\sigma = \exp(\log \sigma^2/2)$ and $z = \mu + \sigma\varepsilon$:

$$\begin{aligned} m = 1 : \quad \sigma &= \exp(-0.5/2) = \exp(-0.25) \approx 0.7788. \\ z &= 0.3 + 0.7788 \times (-0.2) = 0.3 - 0.1558 = 0.1442. \end{aligned}$$

$$\begin{aligned} m = 2 : \quad \sigma &= \exp(0.2/2) = \exp(0.1) \approx 1.1052. \\ z &= -0.4 + 1.1052 \times 0.5 = -0.4 + 0.5526 = 0.1526. \end{aligned}$$

(b)

$$R^{(1)} = -[1 \cdot \log 0.7 + 0 \cdot \log(1 - 0.7)] = -\log 0.7 \approx \mathbf{0.3567}.$$

$$R^{(2)} = -[0 \cdot \log 0.3 + 1 \cdot \log(1 - 0.3)] = -\log 0.7 \approx \mathbf{0.3567}.$$

Both are $-\log 0.7$ because sample 1 has $x = 1$ and the model predicts 0.7 (off by 0.3 from the on side), while sample 2 has $x = 0$ and the model predicts 0.3 (off by 0.3 from the off side). The magnitude of the error is the same in both cases, so the BCE penalty is identical.

(c) KL terms, using $\sigma^{2(1)} = \exp(-0.5) \approx 0.6065$ and $\sigma^{2(2)} = \exp(0.2) \approx 1.2214$:

$$D_{\text{KL}}^{(1)} = \frac{1}{2}(0.6065 + 0.09 - (-0.5) - 1) = \frac{1}{2}(0.1965) \approx \mathbf{0.0983}.$$

$$D_{\text{KL}}^{(2)} = \frac{1}{2}(1.2214 + 0.16 - 0.2 - 1) = \frac{1}{2}(0.1814) \approx \mathbf{0.0907}.$$

(d)

$$\mathcal{L} = \frac{1}{2}[(0.3567+0.0983)+(0.3567+0.0907)] = \frac{1}{2}(0.4550+0.4474) = \frac{1}{2}(0.9024) \approx \mathbf{0.4512}.$$

We average over M (rather than sum) so that the loss value and gradient magnitude stay the same regardless of batch size. If we summed, a batch of 200 would produce gradients 100 times larger than a batch of 2, forcing a different learning rate for every batch size. Averaging keeps the scale consistent.